Evaluación comparativa de herramientas de inteligencia artificial generativa para la auditoría de seguridad de smart contracts

Análisis experimental de ChatGPT, Gemini y Claude Code frente a técnicas tradicionales de detección de vulnerabilidades en contratos Solidity



Universitat Oberta de Catalunya

Diego Rodríguez Pérez

Máster en Ciberseguridad y Privacidad

Área: Ciberseguridad / Smart contracts e Inteligencia Artificial

Nombre Tutor/a de TF

Javier Rodríguez Fernández

Profesor/a responsable de la asignatura

Víctor García Font

Fecha Entrega 07/10/2026



Esta obra está sujeta a una licencia de Reconocimiento-NoComercial-SinObraDerivada 3.0 España de Creative Commons

FICHA DEL TRABAJO FINAL

Título del trabajo:	Evaluación comparativa de herramientas de inteligencia artificial generativa para la auditoría de seguridad de smart contracts
Nombre del autor:	Diego Rodríguez Pérez
Nombre del director/a:	Javier Rodríguez Fernández
Nombre del PRA:	Víctor García Font
Fecha de entrega (mm/aaaa):	10/2026
Titulación o programa:	Máster en Ciberseguridad y Privacidad
Área del Trabajo Final:	Ciberseguridad / Inteligencia Artificial aplicada a smart contracts
Idioma del trabajo:	Castellano
Palabras clave	Smart contracts; Large Language Models; Vulnerability detection

Resumen del Trabajo

Los smart contracts permiten ejecutar de forma automática operaciones sobre redes blockchain y pueden gestionar activos digitales de elevado valor. Debido a su carácter difícilmente modificable tras el despliegue, una vulnerabilidad puede producir consecuencias económicas y de seguridad relevantes. Este Trabajo Final de Máster propone evaluar la capacidad de herramientas de inteligencia artificial generativa, concretamente ChatGPT, Gemini y Claude Code, para identificar vulnerabilidades en smart contracts desarrollados en Solidity. Se diseñará un entorno experimental reproducible utilizando contratos con vulnerabilidades previamente conocidas procedentes de benchmarks públicos. Las herramientas se someterán a un procedimiento común de auditoría y sus resultados se compararán entre sí y con analizadores especializados como Slither y Mythril. La evaluación considerará métricas como precision, recall, F1-score, falsos positivos, falsos negativos y consistencia, junto con un análisis cualitativo de las explicaciones y recomendaciones generadas. El objetivo final es determinar hasta qué punto estas herramientas pueden utilizarse como apoyo en una auditoría de smart contracts, identificar sus limitaciones y proponer un procedimiento híbrido que combine inteligencia artificial generativa con técnicas convencionales de análisis.

Abstract

Smart contracts automatically execute operations on blockchain networks and may manage high-value digital assets. Because deployed contracts can be difficult to modify, security vulnerabilities may cause significant technical and financial consequences. This Master's Thesis proposes an evaluation of the ability of generative artificial intelligence tools, specifically ChatGPT, Gemini and Claude Code, to identify vulnerabilities in Solidity smart contracts. A reproducible experimental environment will be designed using contracts with previously known vulnerabilities obtained from public benchmarks. The selected tools will be evaluated under a common auditing procedure, and their results will be compared both with each other and with specialized analyzers such as Slither and Mythril. The evaluation will consider precision, recall, F1-score, false positives, false negatives and consistency, together with a qualitative analysis of the explanations and remediation recommendations produced. The final objective is to determine to what extent generative AI can support smart-contract security auditing, identify its main limitations, and propose a hybrid workflow combining generative AI with conventional analysis techniques.

Índice

1. Introducción 2
1.1. Contexto y justificación del Trabajo 2
1.2. Objetivos del Trabajo 5
1.3. Impacto en sostenibilidad, ético-social y de diversidad 7
1.4. Enfoque y método seguido 9
1.5. Planificación del Trabajo 11
1.6. Breve sumario de productos obtenidos 17
1.7. Breve descripción de los otros capítulos de la memoria 17
1.8. Declaración sobre el uso de inteligencia artificial generativa 18
2. Materiales y métodos 18
3. Resultados 19
4. Conclusiones y trabajos futuros 19
5. Glosario 20
6. Bibliografía 21
7. Anexos 22

Lista de figuras

Figura 1. Planificación temporal del TFM (calendario oficial del aula). 12

Lista de tablas

Tabla 1. Síntesis de los estudios revisados y su relación con este trabajo. 4
Tabla 2. Objetivos específicos y criterios de éxito. 6
Tabla 3. Impactos en sostenibilidad, ética y diversidad, y medidas de gestión. 8
Tabla 4. Estimación del esfuerzo y del calendario del TFM. 11
Tabla 5. Distribución orientativa de las horas por fase. 11
Tabla 6. Tareas, horas, dependencias y entregables (numeración del Gantt). 13
Tabla 7. Trazabilidad entre objetivos, tareas, entregables y validación. 13
Tabla 8. Riesgos principales, probabilidad, impacto y mitigación. 14
Tabla 9. Estimación de costes del TFM. 15
Tabla 10. Estructura prevista para la comparación cuantitativa de las herramientas. 17

1. Introducción

Los smart contracts constituyen uno de los principales componentes sobre los que se construyen aplicaciones descentralizadas basadas en blockchain. Permiten definir y ejecutar automáticamente reglas y operaciones sin necesidad de intermediarios, pudiendo gestionar activos digitales y ejecutar transacciones de forma autónoma.

Esta característica aporta ventajas relevantes, pero también introduce riesgos de seguridad específicos. Una vulnerabilidad presente en el código de un smart contract puede provocar comportamientos no previstos, accesos no autorizados o pérdidas económicas. Además, una vez desplegado un contrato en una red blockchain, su modificación puede resultar compleja o incluso imposible, por lo que la detección temprana de errores adquiere especial importancia (Atzei et al., 2017).

Tradicionalmente, la auditoría de smart contracts se apoya en revisión manual y en herramientas especializadas de análisis estático, ejecución simbólica, fuzzing y otras técnicas automatizadas. Trabajos previos muestran que estas herramientas presentan diferencias significativas de cobertura y pueden producir tanto falsos positivos como falsos negativos (Durieux et al., 2020). Slither, por ejemplo, es un framework de análisis estático específicamente diseñado para contratos Ethereum y Solidity (Feist et al., 2019).

Paralelamente, la evolución de los grandes modelos de lenguaje ha abierto nuevas posibilidades para el análisis de código fuente. Herramientas de inteligencia artificial generativa como ChatGPT, Gemini o Claude Code pueden interpretar código, explicar su funcionamiento, sugerir errores y proponer modificaciones. Estudios recientes indican que estos modelos pueden detectar determinadas vulnerabilidades de smart contracts, aunque su precisión y robustez varían según el tipo de vulnerabilidad y el contexto (Chen et al., 2025).

El presente Trabajo Final de Máster pretende estudiar experimentalmente hasta qué punto estas herramientas pueden utilizarse como apoyo en la auditoría de seguridad de smart contracts desarrollados en Solidity. El objetivo no es asumir que la IA generativa pueda sustituir una auditoría profesional, sino determinar de forma objetiva qué capacidades ofrece, qué limitaciones presenta y en qué escenarios puede aportar valor dentro de un proceso de análisis de seguridad.

1.1. Contexto y justificación del Trabajo

La seguridad de los smart contracts constituye un problema relevante dentro del ecosistema blockchain. Estos programas pueden gestionar activos económicos y ejecutar operaciones de manera automática, por lo que un fallo de programación puede tener consecuencias especialmente graves. La literatura académica ha documentado ataques y vulnerabilidades específicas del entorno Ethereum, entre ellas reentrancia, errores de control de acceso, dependencias temporales, problemas en llamadas externas y fallos lógicos (Atzei et al., 2017).

Para detectar estas vulnerabilidades se emplean revisiones manuales y herramientas automáticas. Sin embargo, no existe una técnica única capaz de garantizar la ausencia completa de errores. Un estudio empírico sobre decenas de miles de contratos Ethereum mostró diferencias importantes entre nueve herramientas automáticas y evidenció problemas de cobertura y falsos positivos (Durieux et al., 2020). Esta situación justifica la búsqueda de mecanismos complementarios de análisis.

La aparición de modelos de lenguaje capaces de razonar sobre código introduce una aproximación diferente. En lugar de limitarse a patrones predefinidos, estos sistemas

pueden interpretar fragmentos de código y generar explicaciones en lenguaje natural. No obstante, una respuesta convincente no implica que el análisis sea correcto. En seguridad, un falso negativo puede ocultar una vulnerabilidad real y un exceso de falsos positivos puede reducir considerablemente la utilidad práctica de la herramienta. Chen et al. (2025) observaron precisamente una elevada capacidad de detección de ChatGPT en algunos escenarios, pero también limitaciones de precisión, variabilidad entre vulnerabilidades y problemas de robustez.

Problema identificado

Las evaluaciones disponibles son heterogéneas y no permiten saber con claridad hasta qué punto las herramientas de inteligencia artificial generativa de uso común pueden emplearse de forma fiable como apoyo a la auditoría de smart contracts, en qué categorías de vulnerabilidad funcionan mejor y con qué tasa de falsos positivos y de consistencia entre ejecuciones.

Estado del arte resumido

La revisión preliminar de la literatura se organiza en tres bloques: evaluaciones de modelos de lenguaje en modo conversacional, sistemas que combinan modelos con análisis estático y evaluaciones recientes con agentes. Se trata de una aproximación inicial, que se ampliará y sistematizará en la PEC2.

Los primeros estudios evaluaron los modelos mediante instrucciones directas sobre el código. David et al. (2023) analizaron 52 contratos DeFi comprometidos y observaron que GPT-4 y Claude identificaban el tipo correcto de vulnerabilidad en 40 casos, pero con una tasa elevada de falsos positivos que obliga a mantener la revisión humana. Chen et al. (2025) encontraron en ChatGPT un recall alto pero una precisión limitada, además de variabilidad entre tipos de vulnerabilidad y problemas de robustez.

Después aparecieron sistemas que estructuran el uso del modelo. GPTLens separa la generación de hallazgos de su discriminación, de modo que el modelo actúa primero como auditor y después como crítico, y eleva del 38,5 % al 76,9 % la proporción de contratos en los que la primera respuesta es una vulnerabilidad real (Hu et al., 2023). GPTScan combina GPT con análisis estático: alcanza un recall superior al 70 %, una precisión superior al 90 % en contratos de tokens y del 57,14 % en proyectos grandes, y detectó nueve vulnerabilidades que habían pasado desapercibidas a auditores humanos (Sun et al., 2024). Una revisión sistemática reciente concluye que las herramientas basadas en modelos de lenguaje disponibles para su evaluación no están en condiciones de sustituir a Slither o Mythril: superan a estos en algunas categorías (por ejemplo, desbordamiento aritmético o dependencia del orden de las transacciones) y quedan por detrás en otras (por ejemplo, reentrancia o llamadas sin comprobar), por lo que recomienda enfoques híbridos (Ince et al., 2025).

La evaluación se ha desplazado después hacia agentes y entornos más realistas. SCONE-bench reúne 405 contratos explotados entre 2020 y 2025: en simulación, diez modelos generaron exploits funcionales para el 51,11 % de ellos (Xiao et al., 2025). EVMbench evalúa agentes en tres modos detectar, parchear y explotar sobre 120 vulnerabilidades procedentes de 40 repositorios de auditorías (Wang et al., 2026). Una reevaluación independiente matiza estos resultados: el ranking de agentes no es estable entre configuraciones, tareas y conjuntos de datos, el entorno de ejecución del agente puede modificar los resultados y los benchmarks miden el recall sin penalizar los falsos positivos, por lo que la utilidad práctica puede estar sobreestimada (Peng et al., 2026). La Tabla 1 resume los estudios y su relación con este trabajo.

Estudio	Herramientas y datos	Hallazgo principal	Relación con este TFM
David et al. (2023)	GPT-4 y Claude; 52 contratos DeFi comprometidos	Identifican el tipo correcto en 40 casos, con muchos falsos positivos	Justifica medir los falsos positivos y mantener la revisión humana
Chen et al. (2025)	ChatGPT	Recall alto, precisión limitada y variabilidad entre vulnerabilidades	Justifica medir precisión y consistencia
Hu et al. (2023)	GPT (GPTLens)	Pasa del 38,5 % al 76,9 % de aciertos con un esquema auditor-crítico	El protocolo de uso afecta al resultado: se define un prompt común
Sun et al. (2024)	GPT y análisis estático (GPTScan)	Recall > 70 %; precisión > 90 % en tokens y 57,14 % en proyectos grandes	Respalda el enfoque híbrido y la comparación con analizadores
Ince et al. (2025)	Revisión sistemática; herramientas LLM frente a Slither y Mythril	No sustituyen a las herramientas tradicionales; mejores en unas categorías y peores en otras	Justifica comparar por categoría con una línea base
Xiao et al. (2025)	Agentes; SCONE-bench (405 contratos)	Exploits funcionales en el 51,11 % de los casos en simulación	Muestra la relevancia del modo agente y el riesgo de uso dual
Wang et al. (2026); Peng et al. (2026)	EVMbench y su reevaluación	Rankings inestables; el entorno del agente influye; no se penalizan los falsos positivos	Obliga a documentar versiones y modo de acceso y a medir falsos positivos

Tabla 1. Síntesis de los estudios revisados y su relación con este trabajo.

Del conjunto se desprenden tres conclusiones que condicionan el diseño del TFM. Primera, los resultados dependen de la versión del modelo, del prompt y del entorno de ejecución, por lo que deben registrarse. Segunda, el recall por sí solo no basta: son necesarias métricas de precisión, falsos positivos y consistencia. Tercera, los benchmarks públicos y anteriores a los modelos evaluados, como SmartBugs, pueden solaparse con los datos de entrenamiento, lo que obliga a interpretar los resultados con cautela y a controlar este riesgo.

Vacío identificado y aportación prevista

A partir de esta revisión preliminar se identifica el siguiente vacío. Los estudios revisados evalúan con frecuencia una sola familia de modelos o versiones ya superadas, documentan de forma desigual el prompt y el entorno de ejecución, y los benchmarks más recientes miden el recall sin penalizar los falsos positivos (Peng et al., 2026). Falta, por tanto, una comparación homogénea y reproducible de las herramientas que utilizan en la práctica desarrolladores y auditores ChatGPT y Gemini en modo conversacional y Claude Code en modo agente frente a una línea base de análisis estático (Slither y Mythril; Feist et al., 2019; ConsenSys, 2023), con métricas de precisión, falsos positivos y consistencia y con un control explícito del riesgo de contaminación de datos. Este vacío se ha identificado en una revisión inicial y se confirmará en la PEC2.

El trabajo propone cubrirlo mediante una comparación controlada: los mismos contratos, las mismas instrucciones y las mismas métricas para todas las herramientas, contrastadas con vulnerabilidades conocidas y con analizadores especializados. Esto permitirá valorar su posible uso como mecanismo de apoyo sin basarse en ejemplos aislados ni en la calidad aparente de las respuestas.

La relevancia del problema es además creciente. El coste de la explotación automatizada de contratos se reduce con rapidez: en SCONE-bench, el número

mediano de tokens necesarios para generar un exploit exitoso descendió un 70,2 % entre cuatro generaciones de modelos de una misma familia (Xiao et al., 2025). Quienes defienden necesitan, por tanto, evidencia sólida sobre lo que pueden esperar de estas mismas tecnologías.

Motivación

La elección del tema surge del interés por dos ámbitos con una evolución especialmente rápida: la ciberseguridad y la inteligencia artificial generativa. La posibilidad de utilizar herramientas capaces de analizar código mediante lenguaje natural plantea oportunidades para complementar las tareas de revisión de seguridad, pero también obliga a evaluar críticamente su fiabilidad antes de considerar su aplicación en escenarios reales.

Alcance

El alcance se delimita para garantizar que el trabajo sea viable dentro de un TFM y que los resultados puedan ser evaluados de forma reproducible.

• Smart contracts escritos en Solidity y compatibles con Ethereum/EVM.

• Análisis de código fuente y detección de vulnerabilidades previamente conocidas.

• ChatGPT, Gemini y Claude Code como principales herramientas de IA generativa.

• Comparación con herramientas especializadas, principalmente Slither y Mythril.

• Uso de benchmarks públicos, con SmartBugs como candidato principal por facilitar la ejecución y comparación reproducible de analizadores de Solidity (Ferreira et al., 2020).

• Evaluación cuantitativa mediante precision, recall, F1-score, falsos positivos y falsos negativos, junto con un análisis cualitativo de las explicaciones generadas.

Quedarán fuera del alcance los ataques contra contratos desplegados en redes productivas, la generación o ejecución de exploits operativos, la reparación automática del código, las auditorías de sistemas reales sin autorización, el entrenamiento de un modelo propio, el análisis exhaustivo de todos los lenguajes blockchain y el desarrollo de una plataforma comercial de auditoría.

1.2. Objetivos del Trabajo

A partir del problema y del vacío identificados se establecen un objetivo principal, una hipótesis y cinco objetivos específicos. Los objetivos se formulan como resultados que puedan evaluarse al finalizar el trabajo y cada uno lleva un criterio de éxito verificable; las acciones necesarias para alcanzarlos se tratan como tareas en el apartado 1.5.

Objetivo principal

Evaluar experimentalmente la capacidad de diferentes herramientas de inteligencia artificial generativa para detectar vulnerabilidades de seguridad en smart contracts, determinando su precisión, consistencia, limitaciones y posible complementariedad con herramientas tradicionales de auditoría.

Hipótesis de trabajo

Las herramientas de IA generativa igualan o superan el recall de Slither y Mythril en algunas categorías de vulnerabilidad, pero con menor precisión y consistencia, por lo que sirven de apoyo y no de sustituto de la auditoría especializada.

Objetivos específicos

Los objetivos específicos se derivan del objetivo principal y permiten comprobar su cumplimiento al finalizar el TFM. La Tabla 2 recoge cada objetivo con su criterio de éxito; su relación con las tareas, los entregables y la validación se detalla en la Tabla 7.

ID	Objetivo específico	Criterio de éxito
OE1	Caracterizar el estado actual de las técnicas de auditoría de smart contracts y del uso de modelos de lenguaje para detectar vulnerabilidades, identificando sus capacidades y limitaciones.	Estado del arte y matriz comparativa de enfoques, revisados con la tutoría (PEC2).
OE2	Construir un corpus representativo de smart contracts Solidity con vulnerabilidades conocidas que actúe como ground truth, controlando el riesgo de contaminación de datos.	Entre 30 y 40 contratos con vulnerabilidad, localización y fuente documentadas, que cubran al menos cinco categorías, con resultados separados por origen del caso.
OE3	Definir y aplicar un protocolo experimental reproducible a ChatGPT, Gemini, Claude Code, Slither y Mythril en condiciones comparables.	Protocolo versionado (prompt, versión, fecha y modo de acceso); todas las herramientas ejecutadas sobre el 100 % del corpus; al menos tres repeticiones por caso en las herramientas de IA, o en una muestra si el coste lo exige.
OE4	Cuantificar y comparar el rendimiento de cada herramienta (precision, recall, F1, falsos positivos, falsos negativos y consistencia) por categoría de vulnerabilidad y evaluar la calidad de sus explicaciones, severidades y mitigaciones.	Tablas de métricas completas por herramienta y categoría; hallazgos clasificados con criterios documentados y segunda revisión, en un momento posterior, de al menos el 20 % de ellos; rúbrica cualitativa definida antes de ejecutar las pruebas.
OE5	Determinar las ventajas, limitaciones y riesgos de la IA generativa en la auditoría de smart contracts y proponer un procedimiento híbrido.	Hipótesis aceptada, rechazada o matizada con evidencias; procedimiento trazable a los resultados; impactos del apartado 1.3 revisados.

Tabla 2. Objetivos específicos y criterios de éxito.

Producto mínimo viable y objetivos secundarios

Para proteger el proyecto frente a imprevistos se distingue entre el producto mínimo viable y los objetivos secundarios. El producto mínimo viable es la versión más reducida del trabajo que sigue permitiendo responder al objetivo principal: un corpus de al menos 30 contratos, tres herramientas de IA (ChatGPT, Gemini y Claude Code, que representan el modo conversacional y el modo agente), Slither como línea base, al menos tres ejecuciones por caso en una muestra y las métricas principales (precision, recall y F1). Gemini y Mythril forman parte del plan base, pero podrían retirarse sin comprometer el producto mínimo viable si se materializan los riesgos de coste, compatibilidad o disponibilidad descritos en la Tabla 8.

Se consideran objetivos secundarios, que se abordarán solo si el calendario lo permite:

• ampliar el corpus hasta unos 50 contratos;

• evaluar el efecto de aportar contexto adicional (documentación o especificación) además del código;

• comparar el modo conversacional y el modo agente sobre un mismo modelo, cuando sea posible.

Preguntas de investigación

Para guiar el desarrollo experimental se plantean cuatro preguntas de investigación, cada una asociada a los objetivos específicos que contribuye a responder. Servirán para estructurar las pruebas y relacionar los resultados con los objetivos del TFM.

• **RQ1 (OE3, OE4).** ¿Qué capacidad de detección y qué precisión tienen ChatGPT, Gemini y Claude Code sobre vulnerabilidades conocidas, y cómo varían según el tipo de vulnerabilidad?

• **RQ2 (OE4).** ¿Cómo se comparan sus resultados con los de analizadores especializados como Slither y Mythril?

• **RQ3 (OE3, OE4).** ¿Hasta qué punto son consistentes los resultados al repetir una misma auditoría?

• **RQ4 (OE4, OE5).** ¿Qué valor adicional aportan las explicaciones y las recomendaciones de mitigación, y resulta más adecuado utilizar estas herramientas de forma independiente o como complemento de una auditoría tradicional?

1.3. Impacto en sostenibilidad, ético-social y de diversidad

El proyecto analiza su impacto desde las tres dimensiones de la competencia transversal de Compromiso Ético y Global (Universitat Oberta de Catalunya [UOC], 2022): sostenibilidad, comportamiento ético y responsabilidad social, y diversidad y derechos humanos. El análisis se revisará en el diseño, durante el desarrollo y en las conclusiones del trabajo, y se resume en la Tabla 3.

Sostenibilidad

El trabajo presenta un impacto positivo potencial al estudiar mecanismos que permitan detectar vulnerabilidades en fases tempranas del desarrollo, reduciendo el riesgo de incidentes y el coste de su corrección posterior. Se relaciona principalmente con el ODS 9, Industria, innovación e infraestructura (Naciones Unidas, 2015), por su contribución potencial a infraestructuras digitales más seguras y resilientes; esta contribución es indirecta, porque el trabajo aporta evidencia sobre una técnica de apoyo a la seguridad y no desarrolla infraestructura. Como impacto negativo limitado, relacionado con el ODS 13, Acción por el clima, debe considerarse el consumo energético asociado a la inferencia de los modelos y a la repetición de ejecuciones. Para reducirlo se definirá previamente el conjunto de pruebas, se limitarán las ejecuciones a las necesarias para obtener resultados representativos, se reutilizarán las respuestas ya registradas y se anotarán el número de consultas y de tokens como indicador aproximado de consumo.

Comportamiento ético y responsabilidad social

El principal riesgo ético consiste en interpretar una respuesta generada por IA como evidencia definitiva de que un sistema es seguro. Para evitarlo, los resultados se contrastarán con un ground truth conocido y con analizadores adicionales, se documentarán los falsos positivos y los falsos negativos y se mantendrá la supervisión humana sobre las conclusiones.

Un segundo riesgo es el uso dual: los hallazgos y los prompts podrían facilitar ataques. Por ello no se realizarán pruebas sobre contratos desplegados sin autorización, no se generarán ni ejecutarán exploits contra sistemas reales y, si se detectase una vulnerabilidad nueva en un contrato real, se notificaría a sus responsables antes de cualquier difusión (divulgación coordinada). Tampoco se introducirán datos sensibles o

confidenciales en servicios externos de IA: el código analizado procederá de benchmarks públicos.

Por último, ChatGPT, Gemini y Claude Code son a la vez objeto de estudio y, en algunos casos, herramientas de apoyo a la redacción, lo que puede introducir sesgo. Este uso se declara en el apartado 1.8 y ninguna herramienta de IA se utilizará como único criterio para clasificar los resultados experimentales. El trabajo se relaciona además con el ODS 16, Paz, justicia e instituciones sólidas, de forma indirecta y limitada, por su posible contribución a reducir el fraude y el robo de activos digitales (meta 16.4).

Diversidad y derechos humanos

El trabajo no tomará decisiones automatizadas sobre personas ni tratará atributos personales protegidos, por lo que su impacto directo en cuestiones de género es nulo o no aplicable. Sí se consideran otros aspectos. El acceso a auditorías profesionales y a herramientas de IA de pago es desigual: el efecto sobre equipos pequeños o con pocos recursos puede ser positivo si se demuestra que herramientas accesibles aportan valor, o negativo si se generan expectativas no fundadas. Además, los modelos pueden presentar diferencias de rendimiento derivadas de sus datos de entrenamiento, del idioma de los comentarios y la documentación o de la estructura del código. Para mitigarlo se documentará el coste de las herramientas utilizadas, se publicarán el protocolo y el corpus en un repositorio con licencia abierta cuando las licencias de origen lo permitan, las conclusiones se limitarán al conjunto de pruebas evaluado y la memoria utilizará un lenguaje inclusivo.

Dimensión	Impacto previsto	Tipo	Medida de gestión
Sostenibilidad	Infraestructuras digitales más seguras y menos incidentes evitables (ODS 9)	Positivo, indirecto	Publicar protocolo y resultados con sus límites
Sostenibilidad	Consumo energético de la inferencia y de la repetición de pruebas (ODS 13)	Negativo, limitado	Limitar ejecuciones, reutilizar respuestas registradas y anotar consultas y tokens
Ética y responsabilidad social	Uso dual de hallazgos y prompts	Negativo	Sin exploits ni pruebas sobre sistemas reales; divulgación coordinada
Ética y responsabilidad social	Falsa sensación de seguridad por respuestas convincentes	Negativo	Contraste con ground truth, supervisión humana y límites explícitos en las conclusiones
Ética y responsabilidad social	Confidencialidad del código enviado a servicios externos	Negativo	Usar solo código de benchmarks públicos y advertir del riesgo en las recomendaciones
Ética y responsabilidad social	Posible sesgo por utilizar como asistentes las herramientas evaluadas	Riesgo	Declararlo en el apartado 1.8; la IA no es el único criterio de clasificación
Diversidad y derechos humanos	Acceso desigual a auditoría y a herramientas de pago; sesgos de los modelos	Mixto	Documentar costes, publicar protocolo y corpus si las licencias lo permiten y acotar las conclusiones
Diversidad y derechos humanos	Sin tratamiento de datos personales ni decisiones sobre personas	Neutro	Lenguaje inclusivo en la memoria; no perfilar direcciones de cuenta

Tabla 3. Impactos en sostenibilidad, ética y diversidad, y medidas de gestión.

1.4. Enfoque y método seguido

Para la gestión del proyecto se seguirá un enfoque principalmente secuencial con revisiones iterativas, adecuado para un TFM individual. Las fases presentan dependencias claras fundamentación, diseño experimental, ejecución, análisis y conclusiones , pero al cierre de cada una se realizará una revisión de los entregables y criterios de validación antes de avanzar. Este enfoque permite mantener la trazabilidad entre objetivos, tareas, evidencias y resultados, y facilita incorporar las correcciones derivadas del feedback de las PEC.

Desde el punto de vista técnico, se seguirá una metodología experimental y comparativa, complementada con una revisión bibliográfica inicial. Esta estrategia se considera adecuada porque el objetivo no es únicamente describir las herramientas, sino medir su comportamiento bajo condiciones controladas y comparar objetivamente sus resultados.

Se han considerado cuatro estrategias: una revisión exclusivamente bibliográfica, de bajo coste pero que no genera evidencia propia ni refleja las versiones actuales de las herramientas; el desarrollo de una herramienta propia que combine modelos y análisis estático, de mayor riesgo y alcance, que desplazaría el foco de la evaluación al desarrollo; un estudio cualitativo con auditores profesionales, valioso pero inviable en el plazo disponible; y un estudio experimental comparativo con un protocolo común. Se ha elegido esta última porque aporta evidencia propia y reproducible, es abordable en las 300 horas disponibles y permite controlar el alcance, complementada con la revisión bibliográfica de la Fase 1.

Fase 1. Revisión del estado del arte

Se revisarán publicaciones científicas y documentación técnica sobre vulnerabilidades de smart contracts, herramientas automáticas de auditoría, análisis estático, ejecución simbólica, benchmarks reproducibles y aplicación de modelos de lenguaje a tareas de análisis de código y ciberseguridad. En la PEC1 se presenta un estado del arte resumido (apartado 1.1); esta fase lo amplía y sistematiza en la PEC2.

Fase 2. Selección del conjunto de pruebas

Se seleccionará un subconjunto controlado de contratos con vulnerabilidades previamente conocidas. SmartBugs constituye un candidato adecuado al proporcionar un marco orientado a la ejecución y comparación reproducible de analizadores Solidity (Ferreira et al., 2020). Se plantea inicialmente trabajar con aproximadamente 30-40 contratos, ajustando el número definitivo en función de la distribución del benchmark y del tiempo disponible.

Las categorías consideradas incluirán, al menos, reentrancia, control de acceso, llamadas externas no comprobadas, vulnerabilidades aritméticas y errores de lógica o denegación de servicio.

El riesgo de contaminación de datos los contratos de benchmarks públicos y antiguos pueden formar parte de los datos de entrenamiento de los modelos se gestionará separando, cuando sea posible, los resultados de los contratos más conocidos de los de casos menos difundidos o modificados, e interpretando los resultados con esa cautela.

Fase 3. Diseño del protocolo de auditoría

Las herramientas de IA analizarán los contratos bajo condiciones lo más equivalentes posible. Se definirá un prompt de auditoría común, se utilizará el mismo código fuente y se solicitará una salida estructurada que incluya vulnerabilidad detectada, localización, severidad, explicación y recomendación. Se registrarán la herramienta, el modelo o versión, la fecha, el prompt y la respuesta completa. ChatGPT y Gemini se utilizarán en modo conversacional y Claude Code en modo agente siempre que se cuente con la capacidad, en su defecnto en modo conversacional. Como el entorno de ejecución del agente puede modificar los resultados (Peng et al., 2026), se registrará el modo de acceso de cada herramienta, se utilizará el mismo código y el mismo prompt y se interpretará la comparación teniendo en cuenta esta diferencia.

Fase 4. Ejecución de pruebas

Cada contrato será analizado mediante ChatGPT, Gemini y Claude Code. Como línea base se utilizarán analizadores tradicionales, principalmente Slither y Mythril. No se pretende demostrar cuál es la mejor herramienta de forma absoluta, sino comparar su comportamiento dentro del entorno experimental definido.

Fase 5. Evaluación cuantitativa

Los resultados se contrastarán con la vulnerabilidad real conocida. Se clasificarán como True Positive (TP), False Positive (FP), False Negative (FN) y, cuando el diseño del dataset lo permita, True Negative (TN). Se calcularán precisión = TP/(TP+FP), recall = TP/(TP+FN) y F1-score como media armónica entre precision y recall. También se estudiará la consistencia entre ejecuciones y el tiempo aproximado de análisis.

Fase 6. Evaluación cualitativa

Se analizará si cada herramienta identifica correctamente la zona afectada, explica la causa y las consecuencias, propone una mitigación válida, asigna una severidad coherente y evita generar información inexistente o técnicamente incorrecta.

Fase 7. Comparación y conclusiones

Finalmente se construirá una matriz comparativa que integre cobertura, precision, falsos positivos, falsos negativos, consistencia, calidad explicativa, facilidad de utilización y limitaciones. A partir de estos resultados se valorará la viabilidad de un procedimiento híbrido que combine IA generativa con técnicas convencionales de análisis.

Seguimiento, validación y control de cambios

El avance se contrastará de forma periódica con la persona que tutoriza el trabajo (se propone una revisión semanal, de acuerdo con la tutoría), mediante un breve informe de lo realizado, lo previsto y los problemas detectados. Las tareas se gestionarán en un tablero de seguimiento y el código, los prompts, las respuestas y los resultados se versionarán en un repositorio Git. Cada fase se cerrará cuando sus entregables hayan sido revisados: el plan, en la PEC1; el marco de comparación, el corpus y el protocolo piloto, en la PEC2; las campañas ejecutadas con sus hallazgos clasificados, en la PEC3; y la memoria y la presentación, en la PEC4. Cualquier ampliación de alcance se acordará con la tutoría y requerirá retirar una tarea de valor equivalente.

1.5. Planificación del Trabajo

La planificación se ha construido a partir de la carga académica de la asignatura y del calendario oficial del semestre. El TFM tiene una carga de 12 créditos ECTS y se desarrolla entre el 23/09/2026 y el 03/02/2027, un periodo de 19 semanas de calendario. Tomando como referencia 25 horas de trabajo por crédito ECTS, la carga total estimada es de 300 horas, equivalente a una dedicación media aproximada de 15,8 horas semanales. La mayor parte del esfuerzo se concentra antes de la entrega de la memoria final del 07/01/2027; la presentación se entrega el 14/01/2027 y la defensa está prevista el 22/01/2027, reservando el periodo posterior para correcciones y cierre.

La Tabla 4 permite comprobar que el esfuerzo previsto es compatible con el calendario y que el Gantt no representa únicamente fechas, sino una distribución razonada de la carga de trabajo. La estimación se revisará si durante el desarrollo aparecen restricciones profesionales o personales que obliguen a replanificar.

Elemento	Estimación
Créditos ECTS del TFM	12 ECTS
Horas de referencia por crédito	25 h/ECTS
Carga total estimada	300 horas
Periodo de la asignatura	23/09/2026 - 03/02/2027
Semanas de calendario	19 semanas
Dedicación media estimada	15,8 h/semana

Tabla 4. Estimación del esfuerzo y del calendario del TFM.

Como referencia de control, las 300 horas se distribuyen de forma orientativa entre las principales fases del proyecto, como muestra la Tabla 5. Esta distribución podrá reajustarse manteniendo la carga total y los hitos oficiales de entrega.

Fase	Horas	Peso aprox.
Definición, planificación y revisión inicial	35	12 %
Estado del arte y diseño experimental	55	18 %
Preparación y ejecución de pruebas	95	32 %
Normalización, métricas y análisis	55	18 %
Redacción, revisión y memoria final	40	13 %
Presentación, defensa y cierre	20	7 %
Total	300	100 %

Tabla 5. Distribución orientativa de las horas por fase.

La dedicación media de 15,8 horas semanales es un valor aritmético que debe contrastarse con la disponibilidad real. Se tendrán en cuenta los festivos y las vacaciones de diciembre, que coinciden con las tareas 4.5 a 4.7 del Gantt (del

18/12/2026 al 05/01/2027). Dentro de cada fase se reserva aproximadamente un 10 % de las horas para correcciones y contingencias; si la disponibilidad real fuese inferior, se aplicarán las medidas de la Tabla 8 y se priorizará el producto mínimo viable.

A partir de esta estimación, el diagrama de Gantt (Figura 1) organiza las tareas y los hitos de las distintas PEC, incorporando las fechas de feedback y las dependencias entre preparación, ejecución, análisis y redacción. La Tabla 6 detalla cada tarea con el objetivo al que contribuye, las horas estimadas, las dependencias y el entregable.

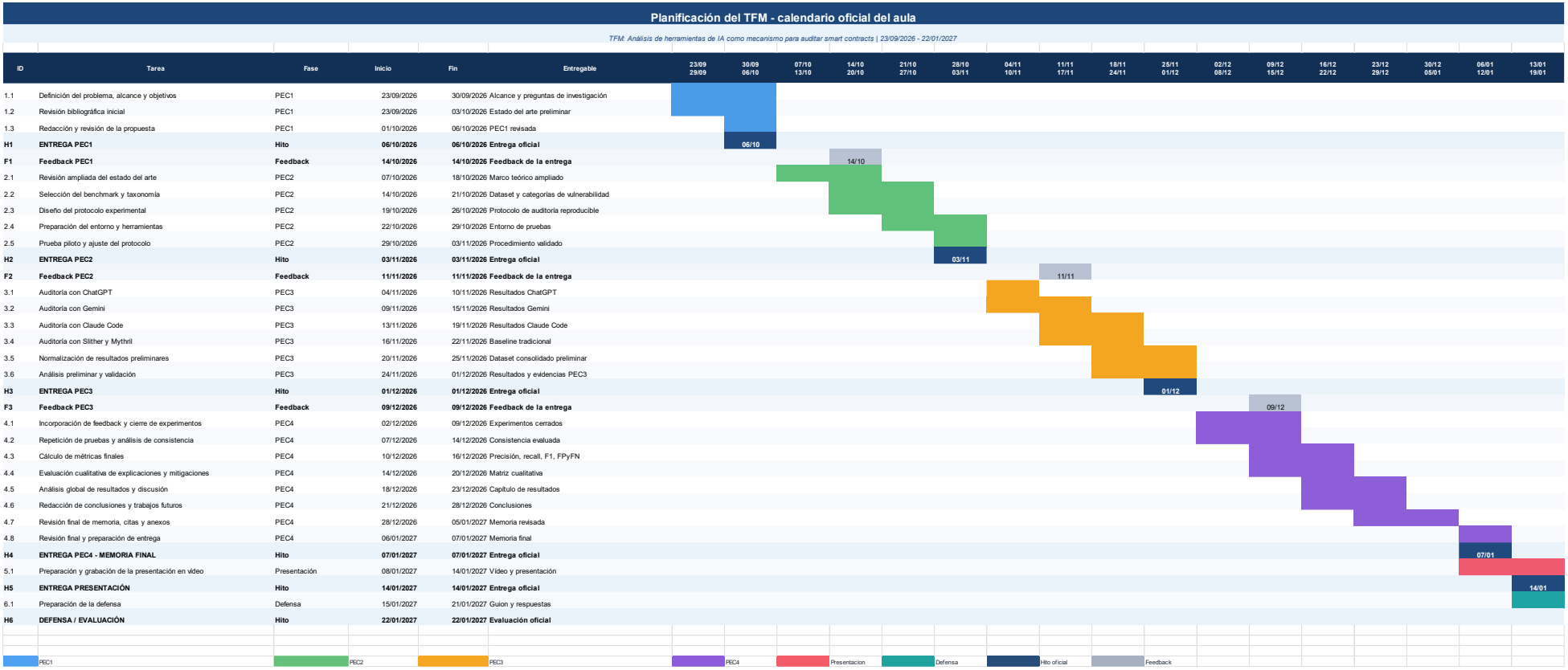


Figura 1. Planificación temporal del TFM (calendario oficial del aula).

ID	Tarea	Obj.	Fechas (2026–27)	h	Depende de	Entregable
1.1	Definición del problema, alcance y objetivos	Todos	23/09–30/09	8		Alcance y preguntas de investigación
1.2	Revisión bibliográfica inicial	OE1	23/09–03/10	14		Estado del arte preliminar
1.3	Redacción y revisión de la propuesta	Todos	01/10–06/10	13	1.1, 1.2	PEC1 revisada
2.1	Revisión ampliada del estado del arte	OE1	07/10–18/10	20	1.3	Marco teórico ampliado y matriz comparativa
2.2	Selección del benchmark y taxonomía	OE2	14/10–21/10	15	2.1 (parcial)	Dataset y categorías de vulnerabilidad
2.3	Diseño del protocolo experimental	OE3	19/10–26/10	20	2.2	Protocolo de auditoría reproducible
2.4	Preparación del entorno y herramientas	OE3	22/10–29/10	18	2.3	Entorno de pruebas
2.5	Prueba piloto y ajuste del protocolo	OE3	29/10–03/11	12	2.4	Procedimiento validado
3.1	Auditoría con ChatGPT	OE3	04/11–10/11	17	2.5	Resultados ChatGPT
3.2	Auditoría con Gemini	OE3	09/11–15/11	17	2.5	Resultados Gemini
3.3	Auditoría con Claude Code	OE3	13/11–19/11	17	2.5	Resultados Claude Code
3.4	Auditoría con Slither y Mythril	OE3	16/11–22/11	14	2.5	Baseline tradicional
3.5	Normalización de resultados preliminares	OE4	20/11–25/11	12	3.1–3.4	Dataset consolidado preliminar
3.6	Análisis preliminar y validación	OE4	24/11–01/12	10	3.5	Resultados y evidencias PEC3
4.1	Incorporación de feedback y cierre de experimentos	OE3, OE4	02/12–09/12	6	3.6	Experimentos cerrados
4.2	Repetición de pruebas y análisis de consistencia	OE4	07/12–14/12	10	4.1	Consistencia evaluada
4.3	Cálculo de métricas finales	OE4	10/12–16/12	7	4.2	Precision, recall, F1, FP y FN
4.4	Evaluación cualitativa de explicaciones y mitigaciones	OE4	14/12–20/12	10	4.2	Matriz cualitativa
4.5	Análisis global de resultados y discusión	OE5	18/12–23/12	12	4.3, 4.4	Capítulo de resultados
4.6	Redacción de conclusiones y trabajos futuros	OE5	21/12–28/12	10	4.5	Conclusiones
4.7	Revisión final de memoria, citas y anexos	Todos	28/12–05/01	10	4.6	Memoria revisada
4.8	Revisión final y preparación de entrega	Todos	06/01–07/01	8	4.7	Memoria final
5.1	Preparación y grabación de la presentación en vídeo	Todos	08/01–14/01	12	4.8	Vídeo y presentación
6.1	Preparación de la defensa	Todos	15/01–21/01	8	5.1	Guion y respuestas

Tabla 6. Tareas, horas, dependencias y entregables (numeración del Gantt).

La Tabla 7 muestra la trazabilidad entre objetivos, tareas, entregables y validación: cada objetivo específico tiene tareas asociadas, un entregable verificable y una forma explícita de comprobar su cumplimiento.

Objetivo	Tareas	Entregables	Validación
OE1	1.2, 2.1	Estado del arte preliminar; marco teórico ampliado y matriz comparativa	Revisión por la tutoría en PEC1 y PEC2
OE2	2.2	Dataset y categorías de vulnerabilidad	Ground truth documentado por contrato; separación por origen revisada con la tutoría
OE3	2.3–2.5, 3.1–3.4, 4.1	Protocolo reproducible, entorno de pruebas, resultados brutos y baseline	Piloto reproducible (tarea 2.5); registro de versiones, fechas y modo de acceso
OE4	3.5, 3.6, 4.2–4.4	Dataset consolidado, métricas finales y matriz cualitativa	Segunda revisión del 20 % de los hallazgos; métricas recalculables desde el repositorio
OE5	4.5, 4.6	Capítulo de resultados y conclusiones	Contraste con la hipótesis; trazabilidad a los resultados; revisión de impactos (apartado 1.3)
Todos	1.1, 1.3, 4.7, 4.8, 5.1, 6.1	PEC1, memoria final, presentación y defensa	Lista de comprobación de coherencia antes de cada entrega

Tabla 7. Trazabilidad entre objetivos, tareas, entregables y validación.

La planificación reserva tiempo específico para validación, repetición de pruebas, incorporación de feedback, revisión de citas y preparación de la presentación y la defensa. Además, se consideran los riesgos principales de la Tabla 8, valorados según su probabilidad e impacto y acompañados de una medida de mitigación.

Riesgo	Prob.	Impacto	Mitigación
Cambio de modelos durante el TFM	Media	Alta	Registrar versión y fecha de cada prueba
Variabilidad de respuestas de los LLM	Alta	Media	Repetir una muestra de pruebas y medir consistencia
Dataset demasiado amplio	Media	Alta	Limitar la muestra a un subconjunto representativo
Resultados ambiguos o difíciles de clasificar	Media	Alta	Definir previamente criterios de TP, FP y FN
Problemas de compatibilidad con contratos antiguos	Media	Media	Usar versiones reproducibles o entornos aislados
Limitaciones de API o coste	Media	Media	Limitar ejecuciones y registrar consumo
Contaminación de datos: los contratos de benchmarks públicos pueden estar en los datos de entrenamiento	Alta	Alta	Separar los resultados por origen del caso, incluir casos modificados o menos difundidos e interpretar con cautela
Disponibilidad personal inferior a la prevista (trabajo, festivos de diciembre)	Media	Alta	Margen del 10 % por fase, priorizar el producto mínimo viable y replanificar con la tutoría
Comparación poco equitativa entre modo conversacional y modo agente	Media	Media	Registrar el modo de acceso, usar el mismo código y prompt y discutir la diferencia en las conclusiones
Rechazo o limitación de respuestas de seguridad por parte de los proveedores de IA	Baja	Media	Formular los prompts con fines defensivos, registrar los rechazos como resultado y recurrir a otra herramienta equivalente

Tabla 8. Riesgos principales, probabilidad, impacto y mitigación.

Los recursos principales serán un equipo informático personal, Python para el tratamiento de resultados, Solidity, SmartBugs, Slither, Mythril, Git/GitHub y las interfaces o APIs de las herramientas de IA. El coste incremental previsto es reducido, como detalla la Tabla 9, y se concentrará, en su caso, en suscripciones o consumo de API; cualquier coste real se registrará durante el desarrollo.

Concepto	Detalle	Coste estimado
Horas de trabajo	300 h de la persona autora, no monetizadas en esta propuesta	0 €
Equipo informático	Equipo personal existente	0 €
Software	Python, Git/GitHub, SmartBugs, Slither, Mythril (código abierto o gratuitos) y herramientas de planificación gratuitas	0 €
Herramientas de IA	Suscripción o crédito de API de ChatGPT, Gemini y Claude Code durante las fases de ejecución; coste incremental nulo si ya se dispone de ellas. Confirmar suscripciones disponibles y contrastar las tarifas vigentes	Previo de confirmar si se cuentan con las licencias
Datos	Benchmarks y repositorios públicos	0 €
Total		**Indeterminado €**

Tabla 9. Estimación de costes del TFM.

1.6. Breve sumario de productos obtenidos

Al encontrarse el proyecto en su fase inicial, todavía no se dispone de resultados experimentales. En esta PEC se dispone de la propuesta, el estado del arte resumido, los objetivos, la metodología, la planificación y el análisis de impactos, y se identifican los productos que se prevé obtener durante el desarrollo:

• Estado del arte ampliado y matriz comparativa sobre IA generativa aplicada a la auditoría de smart contracts.

• Benchmark de contratos y taxonomía de vulnerabilidades utilizada.

• Procedimiento experimental reproducible y colección de prompts.

• Resultados producidos por ChatGPT, Gemini y Claude Code.

• Resultados de Slither y Mythril utilizados como línea base.

• Dataset normalizado de detecciones y matrices de confusión.

• Métricas comparativas y evaluación cualitativa de las explicaciones.

• Propuesta final de metodología híbrida de auditoría.

1.7. Breve descripción de los otros capítulos de la memoria

El capítulo 2, Materiales y métodos, describirá el entorno experimental, las herramientas utilizadas, el conjunto de smart contracts seleccionado, las vulnerabilidades analizadas, los prompts empleados y los criterios de clasificación de resultados.

El capítulo 3, Resultados, presentará los datos obtenidos para cada herramienta e incluirá métricas cuantitativas, tablas comparativas y el análisis cualitativo de las explicaciones generadas.

El capítulo 4, Conclusiones y trabajos futuros, evaluará el cumplimiento de los objetivos, responderá a las preguntas de investigación, contrastará la hipótesis inicial e identificará las principales limitaciones y líneas futuras.

Los capítulos 5, 6 y 7 contendrán respectivamente el glosario de términos y acrónimos, la bibliografía citada y los anexos necesarios para garantizar la trazabilidad y reproducibilidad de las pruebas.

1.8. Declaración sobre el uso de inteligencia artificial generativa

Durante la realización de este Trabajo Final se han utilizado herramientas de inteligencia artificial generativa como apoyo al proceso de elaboración. Durante la fase inicial y la preparación de la PEC 1 se ha utilizado ChatGPT, de OpenAI, para estructurar la propuesta, revisar la redacción, analizar posibles enfoques metodológicos y organizar los objetivos y tareas del proyecto. En la revisión final de la memoria se ha utilizado además Claude, de Anthropic y ChatGPT de OpenAI para contrastarla con la guía y la hoja de referencia de la asignatura, proponer mejoras y impactos y planificación, teniendo dos fuentes y comparado resultados.

Validación. Las herramientas no se han considerado una fuente académica. Las referencias bibliográficas y los datos citados se han contrastado con la fuente original (artículo, informe o documentación oficial). Las estimaciones de horas, fechas y costes se han revisado con criterio propio y se contrastarán con la tutoría.

Valoración crítica. El uso de IA ha acelerado la estructuración y la revisión del documento, pero introduce riesgos de errores factuales, referencias inexistentes y homogeneización del estilo, por lo que todo el contenido generado se ha revisado y adaptado.

Durante las siguientes fases se utilizarán ChatGPT, Gemini y Claude Code adicionalmente como objetos de estudio, al formar parte de las herramientas cuya capacidad de auditoría será evaluada. Este uso es distinto del de asistencia a la redacción: los resultados experimentales se obtendrán con un protocolo documentado, se contrastarán con vulnerabilidades previamente conocidas y con analizadores especializados, y ninguna herramienta de IA se utilizará como único criterio para clasificarlos. Dado que las herramientas asistentes coinciden en parte con las evaluadas, se declara la posible influencia de este hecho.

La selección final del contenido, las decisiones metodológicas, la planificación, la interpretación de los resultados, la revisión de las fuentes y las conclusiones del trabajo son responsabilidad del estudiante.

2. Materiales y métodos

Para la realización del proyecto se empleará un entorno experimental controlado orientado al análisis de smart contracts Solidity. Las herramientas de IA previstas son ChatGPT, Gemini y Claude Code. Como analizadores especializados se utilizarán principalmente Slither y Mythril, junto con Python, Git/GitHub y un entorno Linux para automatizar el tratamiento y la trazabilidad de los resultados.

El conjunto de pruebas se seleccionará a partir de benchmarks públicos con vulnerabilidades etiquetadas. SmartBugs se considera una base especialmente adecuada porque fue diseñado para facilitar la ejecución y comparación de herramientas de análisis sobre contratos Solidity (Ferreira et al., 2020). Cada contrato incluido en el experimento deberá disponer de información suficiente para establecer un ground truth.

El procedimiento experimental consistirá en seleccionar los contratos, registrar su vulnerabilidad conocida, definir un prompt común, ejecutar cada herramienta, conservar las respuestas completas, normalizar las vulnerabilidades reportadas, contrastarlas con el ground truth, clasificarlas como TP/FP/FN/TN y calcular las métricas. Posteriormente se realizará una evaluación cualitativa de la explicación, severidad y mitigación propuesta.

La metodología, el dataset definitivo, las versiones de las herramientas y los criterios exactos de normalización se cerrarán durante las siguientes PEC para garantizar la reproducibilidad del estudio.

3. Resultados

En el momento de elaboración de esta PEC todavía no se han ejecutado las pruebas experimentales, por lo que no se dispone de resultados definitivos. Este apartado se completará durante las fases de desarrollo y validación.

Se prevé obtener para cada herramienta el número de vulnerabilidades detectadas, TP, FP, FN, precision, recall, F1-score, consistencia entre ejecuciones y una valoración cualitativa de las explicaciones y medidas de mitigación. La Tabla 10 se incluye únicamente como estructura prevista y no contiene resultados ficticios.

Métricas previstas. Los valores se completarán después de ejecutar el experimento; no se incluyen resultados estimados o ficticios en esta PEC.

Herramienta	Precision	Recall	F1-score	FP	FN	Consistencia
ChatGPT	Pendiente	Pendiente	Pendiente	Pendiente	Pendiente	Pendiente
Gemini	Pendiente	Pendiente	Pendiente	Pendiente	Pendiente	Pendiente
Claude Code	Pendiente	Pendiente	Pendiente	Pendiente	Pendiente	Pendiente
Slither	Pendiente	Pendiente	Pendiente	Pendiente	Pendiente	N/A
Mythril	Pendiente	Pendiente	Pendiente	Pendiente	Pendiente	N/A

Tabla 10. Estructura prevista para la comparación cuantitativa de las herramientas.

4. Conclusiones y trabajos futuros

En esta fase inicial todavía no es posible formular conclusiones sobre la eficacia de las herramientas, puesto que las pruebas experimentales no se han ejecutado. Las conclusiones finales deberán determinar si la IA generativa permite detectar vulnerabilidades con una precisión útil, qué diferencias existen entre herramientas, qué vulnerabilidades presentan mayor o menor dificultad y qué grado de consistencia muestran los modelos.

También se comparará el rendimiento con Slither y Mythril y se valorará si la capacidad de generar explicaciones proporciona un valor adicional. La hipótesis inicial se aceptará, rechazará o matizará en función de las evidencias obtenidas.

Asimismo, se evaluará si los impactos previstos en el apartado 1.3 (sostenibilidad, ética y diversidad) se han materializado, se han mitigado o han aparecido otros no previstos, y se revisará el cumplimiento de los criterios de éxito de la Tabla 2.

Entre las limitaciones que deberán considerarse se encuentran el tamaño del dataset, la selección de vulnerabilidades, la dependencia respecto a los prompts, los cambios de versión de los modelos y la naturaleza probabilística de sus respuestas.

Como posibles líneas futuras se contemplan la incorporación de nuevos modelos, benchmarks de mayor tamaño, sistemas multiagente, técnicas de Retrieval-Augmented Generation, integración automática entre LLM y analizadores estáticos, estudio de otros lenguajes de smart contracts y evaluación sobre contratos reales previamente auditados.

5. Glosario

A continuación se definen los principales términos y acrónimos utilizados en la memoria.

AI / IA: Inteligencia Artificial.

LLM: Large Language Model.

Blockchain: Registro distribuido en el que las transacciones se almacenan en bloques enlazados criptográficamente.

Smart contract: Programa ejecutado en una blockchain que implementa automáticamente determinadas reglas.

Solidity: Lenguaje de programación utilizado principalmente para desarrollar smart contracts compatibles con Ethereum.

EVM: Ethereum Virtual Machine.

Reentrancy: Vulnerabilidad que permite reentrar en una función antes de que finalice correctamente el estado de una ejecución previa.

Static Analysis: Técnica que analiza código sin necesidad de ejecutarlo.

True Positive (TP): Vulnerabilidad existente correctamente detectada.

False Positive (FP): Vulnerabilidad reportada que no existe según el ground truth.

False Negative (FN): Vulnerabilidad existente que no ha sido detectada.

Precision: Proporción de detecciones positivas que son correctas.

Recall: Proporción de vulnerabilidades existentes que han sido detectadas.

F1-score: Media armónica entre precision y recall.

Ground truth: Información de referencia considerada correcta frente a la que se comparan los resultados.

Prompt: Instrucción proporcionada a un modelo de inteligencia artificial generativa.

Agente de código: Herramienta basada en un modelo de lenguaje que, además de responder, lee ficheros, ejecuta comandos y encadena acciones de forma autónoma.

Línea base (baseline): Herramienta o procedimiento de referencia, aquí los analizadores Slither y Mythril, con el que se comparan los resultados de la IA generativa.

Contaminación de datos: Solapamiento entre los datos de evaluación y los de entrenamiento de un modelo, que puede sobrestimar su rendimiento.

Divulgación coordinada: Práctica de notificar una vulnerabilidad a sus responsables antes de hacerla pública.

6. Bibliografía

Atzei, N., Bartoletti, M., & Cimoli, T. (2017). A survey of attacks on Ethereum smart contracts (SoK). In *Principles of Security and Trust (POST 2017)* (pp. 164–186). Springer. https://doi.org/10.1007/978-3-662-54455-6_8

Chen, C., Su, J., Chen, J., Wang, Y., Bi, T., Yu, J., Wang, Y., Lin, X., Chen, T., & Zheng, Z. (2025). When ChatGPT meets smart contract vulnerability detection: How far are we? *ACM Transactions on Software Engineering and Methodology, 34*(4), Article 100, 1–30. https://doi.org/10.1145/3702973

ConsenSys. (2023). *Mythril: Security analysis tool for EVM bytecode* [Software]. GitHub. Recuperado el 8 de octubre de 2026 de https://github.com/Consensys/mythril

David, I., Zhou, L., Qin, K., Song, D., Cavallaro, L., & Gervais, A. (2023). *Do you still need a manual smart contract audit?* arXiv. https://arxiv.org/abs/2306.12338

Durieux, T., Ferreira, J. F., Abreu, R., & Cruz, P. (2020). Empirical review of automated analysis tools on 47,587 Ethereum smart contracts. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)* (pp. 530–541). https://doi.org/10.1145/3377811.3380364

Feist, J., Grieco, G., & Groce, A. (2019). Slither: A static analysis framework for smart contracts. In *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)* (pp. 8–15). https://doi.org/10.1109/WETSEB.2019.00008

Ferreira, J. F., Cruz, P., Durieux, T., & Abreu, R. (2020). SmartBugs: A framework to analyze Solidity smart contracts. In *35th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (pp. 1349–1352). https://doi.org/10.1145/3324884.3415298

Hu, S., Huang, T., İlhan, F., Tekin, S. F., & Liu, L. (2023). Large language model-powered smart contract vulnerability detection: New perspectives. In *2023 IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)* (pp. 297–306). IEEE. https://arxiv.org/abs/2310.01152

Ince, P., Yu, J., Liu, J. K., & Du, X. (2025). *Generative large language model usage in smart contract vulnerability detection*. arXiv. https://arxiv.org/abs/2504.04685

Naciones Unidas. (2015). *Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible* (A/RES/70/1). https://sdgs.un.org/2030agenda

Peng, C., Wu, L., & Zhou, Y. (2026). *Re-evaluating EVMbench: Are AI agents ready for smart contract security?* arXiv. https://arxiv.org/abs/2603.10795

Sun, Y., Wu, D., Xue, Y., Liu, H., Wang, H., Xu, Z., Xie, X., & Liu, Y. (2024). GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE '24)*. ACM. https://doi.org/10.1145/3597503.3639117

Universitat Oberta de Catalunya. (2022). *¿Cómo incorporar la competencia "Comportamiento ético y global" al Trabajo Final (TF)?* UOC.

Wang, J., Bigger, A., Xu, X., Lin, J. W., Applebaum, A., Patwardhan, T., Yukseloglu, A., & Watkins, O. (2026). *EVMbench: Evaluating AI agents on smart contract security*. arXiv. https://arxiv.org/abs/2603.04915

Xiao, W., Killian, C., Sleight, H., Chan, A., Carlini, N., & Peng, A. (2025, 1 de diciembre). *AI agents find $4.6M in blockchain smart contract exploits*. Anthropic. https://www.anthropic.com/research/smart-contracts

7. Anexos

En la PEC 1 todavía no es necesario incorporar todos los anexos, pero se prevé utilizar esta sección para conservar evidencias y material que permita reproducir el experimento sin sobrecargar el cuerpo principal de la memoria.

Anexo A. Contratos utilizados

Listado de smart contracts incluidos en el conjunto de pruebas, junto con su procedencia y vulnerabilidad conocida.

Anexo B. Clasificación de vulnerabilidades

Tabla con contrato, categoría de vulnerabilidad, localización, severidad y fuente del benchmark.

Anexo C. Prompts utilizados

Texto completo de las instrucciones empleadas con cada herramienta de IA generativa.

Anexo D. Respuestas generadas

Respuestas completas obtenidas mediante ChatGPT, Gemini y Claude Code o referencia al repositorio donde se conserven.

Anexo E. Resultados de analizadores tradicionales

Salidas completas de Slither y Mythril y parámetros de ejecución.

Anexo F. Dataset consolidado

Dataset con contrato, vulnerabilidad real, herramienta, detección reportada, clasificación TP/FP/FN/TN, explicación y mitigación.

Anexo G. Scripts de análisis

Scripts utilizados para normalizar resultados, calcular métricas y generar tablas o gráficos.